

SENTINEL

Operations Guide

A concise reference for running and maintaining the SENTINEL sanctions screening tool. Covers day-to-day operation, architecture, deployment, and troubleshooting.

Version	1.0 - August 2026
Document	Operations Reference

Contents

1	What SENTINEL is	Overview and watchlists covered
2	Day-to-day use	Single search, bulk screening, sidebar
3	Architecture	Code, hosting, local development
4	Deployment	Streamlit Cloud build and update flow
5	Data freshness	Live fetches, caching, snapshots
6	Troubleshooting	Common issues and how to resolve them

1. What SENTINEL is

SENTINEL is a web-based sanctions name-screening tool. It checks a name or a list of names against eight official international watchlists and reports anything that looks like a match, with a confidence score for each result.

Watchlists covered

- OFAC SDN and Non-SDN lists (US Treasury)
- United Nations Consolidated List
- Australian DFAT Consolidated List
- Singapore MAS Designated Lists and Enforcement Actions
- SG TSFA First Schedule (Singapore Terrorism (Suppression of Financing) Act)
- MAS Investor Alert List (unlicensed entities flagged by MAS - an investor-protection list, not a sanctions list)

List data is fetched live from official public sources and refreshed automatically, so screening always runs against the latest information.

2. Day-to-day use

Single search

Type a name on the Individual Search tab and press enter. SENTINEL returns a verdict (FLAGGED, POTENTIAL or CLEAR), the closest matches, and a downloadable PDF of the findings.

Bulk screening

Upload a spreadsheet (Excel, CSV or JSON), select the name column, and run the screening. The app shows a summary table and three download options:

- Full Report (PDF) - one page per screened subject
- Results (CSV) - the summary table
- Match Details (XLSX) - every raw match with source, confidence and remarks

Reading the results

FLAGGED = strong match. POTENTIAL = possible match. CLEAR = no match found. All results are advisory and require human review before any action is taken.

Sidebar

The sidebar shows Data Status (freshness of each list), a Refresh Data Now button, and Settings for the fuzzy threshold, phonetic matching, and the 'Screened by' name.

3. Architecture

Code

Python + Streamlit. Core files: matcher.py (matching engine), report.py (PDF generation), config.py (settings and source URLs), sources/ (one module per watchlist).

Local dev

Clone the repo, install requirements (pip install -r requirements.txt), run streamlit run app.py.

Hosting

Designed for Streamlit Community Cloud. Push to the main branch triggers an automatic rebuild.

Matching

Five strategies: exact, alias, substring, fuzzy (token-set), and phonetic (Double Metaphone). Thresholds live in config.py.

4. Deployment

To deploy, connect the GitHub repository to Streamlit Community Cloud and choose the branch to publish. Every subsequent commit pushed to that branch is built and published automatically (typically 2-3 minutes). The deployed app shows in the Streamlit dashboard under Manage app.

5. Data freshness

On startup the app fetches every watchlist live from its official source, then caches the results for one hour to avoid unnecessary requests. A snapshot of each source is written to the `data/` directory; if a source becomes unavailable, the app falls back to the most recent snapshot and shows a 'cached' status instead of failing.

- Sidebar and Data Status tab show status, record count and fetch time per list.
- Refresh Data Now clears the cache and re-pulls every list immediately.

6. Troubleshooting

- Download buttons appear a few seconds after the table on large runs - this is normal, not an error.
- If a redacted error is shown, open Manage app > Logs in Streamlit to read the full underlying error.
- If the lists look stale, check Data Status in the sidebar and use Refresh Data Now to re-pull.
- Every report records its data version in the provenance table, so outputs can always be traced back.